

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Кнут-Моррис-Пратт

Студентка гр. 4343

Попова Е.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание.

5.1.1 Кнут-Моррис-Пратт

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка – P
- Вторая строка – T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Входные данные:

ab

abab

Выходные данные:

0,2

5.1.2 Кнут-Моррис-Пратт

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка – A
- Вторая строка – B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Входные данные:

defabc

abcdef

Выходные данные:

3

Выполнение работы.

Основные структуры данных

5.1.1:

- pi ($list[int]$) – массив префикс-функции для шаблона P , где $pi[i]$ хранит длину наибольшего собственного префикса подстроки $P[0..i]$, который одновременно является её суффиксом.
- $occurrences$ ($list[int]$) – список индексов, в которых начинаются найденные вхождения шаблона в текст.
- $pattern$ (str) – искомый шаблон P .
- $text$ (str) – исходный текст T .

5.1.2:

- $doubled_A$ (str) – строка, полученная конкатенацией строки A с самой собой ($A + A$).
- A (str) – первая строка (потенциальный циклический сдвиг).
- B (str) – вторая строка (исходная строка).

Основные функции.

- $prefix_function(pattern)$ – вычисляет префикс-функцию для заданного шаблона. Использует два указателя: i (текущий символ) и j (длина текущего совпадающего префикса/суффикса). При несовпадении символов происходит возвращение к значению $pi[j - 1]$.
- $kmp(pattern, text)$ – основная функция поиска. Вычисляет префикс-функцию для шаблона, затем проходит по тексту, поддерживая длину текущего совпадения j . При полном совпадении ($j == m$) индекс добавляется в список $occurrences$, а j сдвигается к $pi[j - 1]$ для поиска последующих вхождений без возврата по тексту.

- `main()` – точка входа программы. Считывает все входные данные пакетно через `sys.stdin.read().split()`, что ускоряет работу с большими объемами. Проверяет равенство длин строк `A` и `B`. Если длины равны, вызывает `kmp(B, A + A)` и выводит полученный индекс, предварительно проверяя, что он строго меньше `len(A)`.

Описание алгоритма

Алгоритм Кнут-Морриса-Пратта (5.1.1):

1. Вычисление префикс-функции: Для шаблона `P` длины `m` создается массив `pi` размера `m`. Алгоритм последовательно обрабатывает символы шаблона, используя уже вычисленные значения для предотвращения избыточных сравнений. Сложность этого этапа – $O(m)$.
2. Поиск в тексте: Проходим по тексту `T` длины `n`. Поддерживаем длину текущего совпавшего префикса шаблона `j`. Если символы `text[i]` и `pattern[j]` не совпадают, мы не сдвигаем указатель `i` назад, а уменьшаем `j` до `pi[j - 1]`. Если символы совпадают, `j` увеличивается. Если `j` достигает `m`, фиксируется вхождение, и `j` обновляется через префикс-функцию. Сложность этапа – $O(n)$.

Поиск циклического сдвига (5.1.2):

Алгоритм базируется на математическом свойстве: строка `A` является циклическим сдвигом строки `B` тогда и только тогда, когда их длины равны, и строка `B` является подстрокой строки `A`, записанной дважды (`A + A`).

Для поиска первого вхождения `B` в конкатенации `A + A` напрямую используется ранее реализованная функция `kmp(B, A + A)`. Общая линейная асимптотика равна $O(|A| + |B|)$.

Оптимизация алгоритма

Для 5.1.1

1. Отсутствие возвратов по тексту: Главное преимущество КМП – указатель по тексту i всегда движется только вперед. Это гарантирует линейное время работы даже на текстах с большим количеством частичных совпадений (например, текст "aaaa...a" и шаблон "aaab").
2. Использование префикс-функции: Позволяет переиспользовать информацию о внутренних совпадениях шаблона, избегая повторных сравнений символов.

Для 5.1.2

1. Ранняя проверка длин: Проверка $\text{len}(A) \neq \text{len}(B)$ выполняется до конкатенации строк. Это сразу отсекает изначально неподходящие варианты, что помогает избежать лишнего выделения памяти.
2. Ограничение индекса: Условие $\text{pos} < \text{len}(A)$ гарантирует, что мы не вернем индекс, выходящий за границы первого экземпляра строки A .

Для 5.1

1. Оптимизация памяти: Замена стандартного списка `list` на `array.array('i')` для хранения префикс-функции.

Оценка сложности

Задачи 5.1.1

Теоретическая сложность:

- Временная сложность: $O(|P| + |T|)$. Вычисление префикс-функции занимает $O(|P|)$, поиск по тексту – $O(|T|)$. Итог: $O(|P| + |T|)$.
- Пространственная сложность: $O(|P| + K)$, где K – количество вхождений. Память требуется для хранения массива pi размера $|P|$ и списка `occurrences`.

Задача 5.1.2

Теоретическая сложность:

- Временная сложность: $O(|A| + |B|)$. Конкатенация строк занимает $O(|A|)$, поиск подстроки в удвоенной строке – $O(|A| + |B|)$.
- Пространственная сложность: $O(|A|)$ для хранения строки `doubled_A`.

Тестирование

Экспериментальные результаты подтверждают корректность работы алгоритма на различных тестовых случаях.

Таблица 1: Тестирование программы 5.1.1

Входные данные	Выходные данные
P="ab", T="abab"	0,2
P="a", T="aaaa"	0,1,2,3
P="abc", T="defdef"	-1
P="aa", T="aaa"	0,1

Таблица 1: Тестирование программы 5.1.2

Входные данные	Выходные данные
defabc abcdef	3
abcdef abcdef	0
abc abcd	-1
ababab bababa	1

Вывод

В ходе выполнения лабораторной работы был успешно реализован алгоритм Кнут-Морриса-Пратта для поиска всех вхождений шаблона в текст. Особенностью алгоритма является использование префикс-функции, которая позволяет избежать возвратов по тексту и гарантирует линейную асимптотику $O(|P| + |T|)$ в худшем случае.

Для задачи 5.1.2 проверка на циклический сдвиг была сведена к поиску подстроки: строка A конкатенируется сама с собой, после чего с помощью написанной функции `kmp` в ней ищется строка B . Такой подход позволил переиспользовать готовый алгоритм КМП и сохранить общую асимптотику $O(|A| + |B|)$.

Теоретическая и практическая оценка сложности подтверждает эффективность метода: алгоритм требует малых затрат памяти ($O(|P|)$ для 5.1.1 и $O(|A|)$ для 5.1.2) и справляется с большими входными данными (до 5 миллионов символов) за доли секунды, что делает его пригодным для использования в системах полнотекстового поиска и биоинформатике.